

I. QUESTION (QUEUE)

Write a function that inserts a new element after the largest element of the queue. Write the function for array implementation. You are not allowed to use any queue methods, just attributes, constructors, getters and setters.

```
1 void insertAfterLargest(int data)
```

II. QUESTION (QUEUE)

Write a function that creates and returns a new queue by removing even indexed elements from the original queue and inserting into the newly created queue. Write the function for both array and linked list implementations. The first node has index 1. You are not allowed to use any queue or linked list methods, just attributes, constructors, getters and setters.

```
1 Queue divideQueue()
```

III. QUESTION (TREE)

Write a recursive method in `TreeNode` class that computes the sum of all keys that are less than X in a binary search tree. You are not allowed to use any tree methods, just attributes, constructors, getters and setters.

```
1 int sumOfTree(int X)
```

IV. QUESTION (TREE)

Write a non-recursive method in `Tree` class that returns the depth of the node containing a given data X in a binary search tree. You are not allowed to use any tree methods, just attributes, constructors, getters and setters.

```
1 int depthOfNode(int X)
```

V. QUESTION (HASHING)

Write a static method that takes an array of integers as a parameter and checks if the array contains any duplicate elements. Your method should run in $O(N)$ time, where N is the size of the array. You are allowed to use any methods and external data structures we learned in the class.

```
1 boolean anyDuplicate(int[] array)
```

VI. QUESTION (HASHING)

Write a method that simplifies a hash table by creating a new hash table containing elements from the original hash table, where

- For single occurrence of a value, copy that value to the new table
- For multiple occurrences of that value, copy that value only once to the new table

Write the function both array and linked list implementations. You are allowed to use linked list and hashing methods.

```
Hash simplify()
```